

test_smpl/

The two shared test runners

test_smpl/ — the two shared test runners

The test applications are no longer here. Every board owns its own, in `boards/<id>/test/` of the architecture project that has that board. How they are laid out and how to run them is [tests-in-aarch32-aarch64.md](#).

This directory kept only what is genuinely shared and genuinely not a test: the two scripts that drive a build.

```
test_smpl/
├─ run-qemu.sh      run a build's *-qemu images under an emulator
└─ run-hw.sh        run one *-hwd image on silicon, through OpenOCD
```

Neither script knows a board or a port. A board's `boards/<id>/{qemu,hw}.sh` supplies the facts and execs one of these; that is the only way they are called.

```
run-qemu.sh <build-test-dir> <qemu-binary> <machine-args...>
```

Runs every `*-qemu.bin` in the directory and reports `PASS` / `SKIP` / `FAIL` from the `RESULT:` line each test prints, then a one-line summary.

variable	meaning
<code>UOS_QEMU_ONLY</code>	run just this application, with its output live on the terminal instead of only in a log — the run you are watching during bring-up
<code>UOS_TEST_SRC_DIR</code>	the board's <code>test/</code> directory, so a test's host-side helper can be found (<code>sd_test/flatfs_tool.py</code> , which seeds the card image)
<code>UOS_QEMU_SHIM</code> , <code>UOS_QEMU_LOAD_ADDR</code>	for a port that cannot be handed straight to <code>-kernel</code> : the AArch32 BCM2837 build, because QEMU's <code>raspi3b</code> starts its Cortex-A53 cores in AArch64, so a 20-line stub drops to AArch32 and jumps to the image

Tests that talk to an SD card get one: `sd_test` a seeded flatfs volume, the others a 4 GiB sparse blank. Both are kept out of git.

A suite is read from its summary, so each run is captured to `.qemu-logs/<app>.log` . `UOS_QEMU_ONLY` streams as well as captures.

```
run-hw.sh <build-test-dir> <app|list> [run-seconds]
```

Halts the cores, enables semihosting, `load_image`, resumes — and watches the semihosted console for the verdict.

variable	meaning
<code>UOS_HW_CFG</code>	the board's OpenOCD config
<code>UOS_HW_CFG_INIT</code>	<code>1</code> when that config is purely declarative and the runner must issue <code>init</code>
<code>UOS_HW_ENTRY</code>	where the image is linked (read from the ELF when possible)
<code>UOS_HW_SPIN_WORDS</code>	<code>__smp_spin</code> words to zero before releasing the secondaries; <code>0</code> skips that stage
<code>UOS_HW_RESUME</code>	<code>cpsr</code> <code>pc</code> <code>entry</code>
<code>UOS_HW_NCPU</code> , <code>UOS_HW_CORES</code> , <code>UOS_HW_TARGET_FMT</code>	how many cores, which are debug targets at load time, and how they are named (<code>bcm2837.cpu%d</code> , <code>rk3506.a7.%d</code>)
<code>UOS_HW_PRELOAD</code>	Tcl to run between halt and load — the Lyra's MMU/cache sanitize, which has to happen before an image is written over the miniloader's page tables
<code>UOS_HW_NM</code> , <code>UOS_HW_READ ELF</code> , <code>UOS_HW_ADAPTER_KHZ</code>	binutils for the ELF, and the probe clock

Pure OpenOCD. No GDB — the board configs set `gdb port disabled` outright.

No redirection. Everything OpenOCD and the board write reaches your terminal as it happens; the tee'd copy in `.hw-logs/<app>.log` exists only so the loop can match `RESULT:`. The script never opens the tty, so your own `tio -b 115200 /dev/ttyACM0` is never fought over.

It refuses a suite. Every run is a `load_image` into RAM over whatever the previous test left there, and neither board has a reset a script can drive. It takes exactly one test; you power-cycle between them. `list` is the default argument and prints what a build has, with each budget.

These two files replace what the predecessor repository kept as `hw.sh` + `hw-olimex.sh` in every test directory of every port — about 200 near-identical lines, 48 files.